

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikrokontroller

Valian Tsaqif Hidayat - 5024241070

2026

1 Pendahuluan

1.1 Latar Belakang

Mikrokontroler bekerja di dalam hampir semua perangkat elektronik, mulai dari mesin cuci sampai sensor IoT. Satu chip mikrokontroler memuat CPU, memori, dan periferal input/output. Sebelum mempelajari protokol komunikasi, RTOS, atau konektivitas nirkabel, kamu perlu menguasai dua hal dasar. Pertama, memprogram mikrokontroler. Kedua, mengakses periferal paling sederhana, yaitu GPIO (General Purpose Input/Output).

Praktikum ini memakai dua board dengan karakter berbeda. STM32 Blackpill berbasis ARM Cortex-M4 dan menerima program lewat ST-Link melalui SWD. ESP32 berbasis Xtensa LX6 dan menerima program lewat kabel USB karena board memiliki chip USB-to-Serial. Kamu memprogram ESP32 dengan dua framework. Arduino menyediakan abstraksi tinggi, sedangkan ESP-IDF menyediakan akses langsung ke fitur Espressif dan FreeRTOS.

Pemrograman perangkat keras menghadirkan tiga masalah yang tidak muncul pada pemrograman perangkat lunak biasa. Pin input yang tidak terhubung ke level tegangan manapun akan floating. Kontak tombol mekanik memantul dan menghasilkan sinyal ganda. Besaran analog seperti intensitas cahaya harus melewati ADC sebelum diproses. Praktikum ini melatih kamu menyiapkan PlatformIO, melakukan flashing ke kedua board, dan memakai GPIO dalam tiga mode, yaitu digital output, digital input, dan analog input.

1.2 Dasar Teori

1.2.1 PlatformIO dan VSCode sebagai Lingkungan Pengembangan

PlatformIO adalah ekosistem pengembangan embedded lintas platform. Kamu memasangnya sebagai ekstensi di Visual Studio Code. PlatformIO membaca berkas `platformio.ini`, lalu mengunduh toolchain, framework, library, dan uploader sesuai isinya. Berkas ini menyimpan empat parameter utama, yaitu `platform`, `board`, `framework`, dan `upload_protocol`.

Perpindahan antar framework cukup mengubah satu baris. Untuk memprogram ESP32 dengan ESP-IDF, kamu mengganti nilai `framework` dari `arduino` menjadi `espidf`. Satu project terdiri atas tiga folder. Folder `src/` menyimpan kode utama, `include/` menyimpan header, dan `lib/` menyimpan library tambahan. Ekstensi Serial Plotter melengkapi lingkungan ini dengan menampilkan data serial dalam bentuk grafik.

1.2.2 STM32 Blackpill dan Framework Arduino

Blackpill (WeAct STM32F401CCU6) adalah development board berbasis ARM Cortex-M4 32-bit dengan FPU. Spesifikasinya mencakup clock hingga 84 MHz, Flash 256 KB, dan SRAM 64 KB. LED onboard terhubung ke pin PC13 dan bersifat active-low, jadi LED menyala saat pin berlogika LOW. Tombol user terhubung ke PA0.

Board ini tidak memiliki programmer onboard. Kamu perlu ST-Link yang terhubung lewat Serial Wire Debug (SWD). SWD memakai dua jalur, yaitu SWDIO untuk data dua arah dan SWCLK untuk clock. Melalui Debug Access Port (DAP) pada inti Cortex-M, ST-Link menulis Flash secara langsung dan menjalankan debugging seperti breakpoint, step, dan inspeksi register.

Praktikum ini memprogram STM32 dengan STM32duino, yaitu lapisan Arduino di atas HAL STMicroelectronics. Fungsi `pinMode()`, `digitalWrite()`, dan `delay()` tetap berlaku. Kamu menulis pin sesuai label port fisiknya, misalnya `PC13`.

1.2.3 ESP32 dan Framework ESP-IDF

ESP32 adalah SoC dengan dua inti Xtensa LX6 berkecepatan hingga 240 MHz, SRAM 520 KB, WiFi 802.11 b/g/n, dan Bluetooth. Board ESP32 DevKit v1 membawa chip bridge USB-to-UART (CP2102 atau CH340) dan rangkaian auto-reset. Dua komponen ini membuat flashing berjalan lewat kabel USB tanpa menekan tombol apa pun. LED onboard umumnya terhubung ke GPIO2.

ESP-IDF (Espressif IoT Development Framework) adalah framework resmi Espressif yang berdiri di atas FreeRTOS. Program dimulai dari fungsi `app_main()`, yang berjalan sebagai task setelah scheduler aktif. Fungsi `vTaskDelay(pdMS_TO_TICKS(ms))` memindahkan task ke status Blocked selama sejumlah tick. Selama task menunggu, CPU menjalankan task lain. Busy-waiting bekerja berbeda karena menghabiskan siklus CPU hanya untuk menunggu.

1.2.4 Konfigurasi Pull-up dan Pull-down Resistor pada Tombol

Pin input digital CMOS memiliki impedansi sangat tinggi. Tanpa sambungan ke level tegangan tertentu, pin berada dalam kondisi floating. Noise dan kapasitansi parasit membuat pembacaannya berganti HIGH dan LOW secara acak.

Resistor pull-up menghubungkan pin ke VCC. Resistor pull-down menghubungkan pin ke GND. Keduanya memberi pin level default yang pasti. Nilai 10 kΩ umum dipakai karena cukup kuat menahan noise dan membatasi arus saat tombol ditekan:

$$I = \frac{3{,}3\ \text{V}}{10\ \text{k}\Omega} = 0{,}33\ \text{mA}. \tag{1}$$

ESP32 menyediakan resistor pull internal sekitar 45 kΩ. Kamu mengaktifkannya dengan `INPUT_PULLUP` atau `INPUT_PULLDOWN`. Pin GPIO34 sampai GPIO39 menjadi pengecualian karena hanya berfungsi sebagai input dan tidak memiliki resistor pull internal.

1.2.5 Debouncing

Kontak logam pada tombol mekanik bersifat elastis. Saat kamu menekan atau melepas tombol, kontak memantul beberapa kali sebelum stabil. Pantulan ini menghasilkan deretan transisi HIGH dan LOW selama ratusan mikrodetik hingga beberapa milidetik. Mikrokontroler membaca pin dalam orde mikrodetik, sehingga setiap pantulan terhitung sebagai event terpisah.

Debouncing berbasis waktu mencatat waktu perubahan terakhir dengan `millis()`. Program menerima status baru hanya jika sinyal stabil lebih lama dari T_{debounce}, umumnya 20 sampai 50 ms. Nilai yang besar menghasilkan pembacaan lebih bersih, tetapi tombol merespons lebih lambat.

1.2.6 ADC sebagai Mode Akses GPIO Ketiga

GPIO berfungsi sebagai digital output, digital input, dan analog input. Mode ketiga memakai ADC (Analog-to-Digital Converter). ESP32 memiliki dua ADC bertipe SAR (Successive Approximation

Register), yaitu ADC1 pada GPIO32 sampai GPIO39 dan ADC2. Driver WiFi memakai ADC2, jadi kamu tidak bisa menggunakannya saat WiFi aktif. Pilih ADC1 untuk praktikum ini.

Pada resolusi default $N = 12$ bit, nilai digital D dan tegangan rekonstruksinya mengikuti persamaan

$$D = \text{round}\left(\frac{V_{in}}{V_{ref}}\left(2^N - 1\right)\right), \qquad V \approx D \cdot \frac{3{,}3}{4095}. \tag{2}$$

Framework Arduino memakai atenuasi default 11 dB, sehingga rentang masukan mendekati 0 sampai 3,3 V. Respons ADC ESP32 tidak linier di ujung rentang, yaitu mendekati 0 V dan di atas sekitar 2,5 V.

Percobaan memakai LDR sebagai pembagi tegangan dengan $R_1 = 10\ \text{k}\Omega$:

$$V_o = V_{in}\,\frac{R_1}{R_{LDR} + R_1}. \tag{3}$$

Cahaya yang makin terang menurunkan R_{LDR}. Akibatnya V_o dan nilai ADC naik.

2 Tugas Pendahuluan

1. **Perbedaan framework Arduino dan ESP-IDF.** Arduino adalah lapisan abstraksi tingkat tinggi dengan struktur `setup()` dan `loop()`. API-nya seragam di banyak board. ESP-IDF adalah framework native Espressif berbasis FreeRTOS. Program dimulai dari `app_main()` dan mengakses driver periferal serta fitur RTOS secara langsung. Pada Arduino-ESP32, `loop()` juga berjalan di dalam task FreeRTOS, tetapi sistem menyembunyikannya dari pengguna.

Tabel 1: Perbandingan framework Arduino dan ESP-IDF

Aspek	**Arduino**	**ESP-IDF**
Titik masuk	`setup()`, `loop()`	`app_main()`
Abstraksi	Tinggi, API seragam	Rendah sampai menengah, khusus ESP32
RTOS	Tersembunyi dan terbatas	Langsung (task, queue, semaphore)
Kelebihan	Mudah dipelajari, library melimpah, kode portabel, cocok untuk prototipe cepat	Kontrol penuh atas WiFi, power management, dan partition. Performa dan determinisme lebih baik. Didukung resmi Espressif
Kekurangan	Kontrol periferal terbatas, ada overhead abstraksi, kurang cocok untuk produk kompleks	Sulit dipelajari, kode lebih panjang, tidak portabel ke board lain

2. **ST-Link pada Blackpill dan USB pada ESP32.** ESP32 DevKit memiliki chip bridge USB-to-Serial (CP2102 atau CH340). Chip ini mengubah sinyal USB dari PC menjadi UART untuk UART0 ESP32. Rangkaian auto-reset memakai sinyal DTR dan RTS untuk mengendalikan pin EN dan GPIO0. Dengan cara ini `esptool` memaksa ESP32 masuk ke bootloader serial di ROM dan menulis Flash tanpa tombol apa pun.

   Blackpill tidak memiliki chip bridge tersebut. Port USB-nya terhubung langsung ke periferal USB OTG milik STM32, dan periferal ini tidak menjalankan bootloader serial secara default. ST-Link melalui SWD menjadi cara paling andal. ST-Link mengakses inti dan memori Flash lewat debug

port dan mendukung debugging. STM32F401 juga menerima program lewat USB DFU dengan menahan tombol BOOT0. Cara ini manual dan tidak mendukung debugging.

3. **Kondisi logika pull-up dan pull-down.** Pada konfigurasi pull-up, resistor menarik pin ke VCC. Pin terbaca HIGH saat tombol dilepas. Saat tombol ditekan, pin tersambung ke GND dengan hambatan jauh lebih kecil daripada resistor, sehingga pin terbaca LOW. Konfigurasi ini bersifat active-low. Konfigurasi pull-down bekerja sebaliknya dan bersifat active-high.

Tabel 2: Kondisi logika pin pada konfigurasi pull-up dan pull-down

Konfigurasi	Tidak ditekan	Ditekan	Sifat
Pull-up (ke VCC, tombol ke GND)	HIGH	LOW	active-low
Pull-down (ke GND, tombol ke VCC)	LOW	HIGH	active-high

4. **Fenomena bouncing dan cara mengatasinya.** Saat kontak tombol bertemu atau berpisah, pegas dan massa kontak bergetar. Sambungan listrik terbuka dan tertutup berulang kali selama beberapa milidetik sebelum stabil. Mikrokontroler membaca setiap pantulan sebagai penekanan terpisah. Ada tiga pendekatan untuk mengatasinya.

   - Hardware debouncing. Filter low-pass RC meredam transisi cepat. Schmitt trigger seperti 74HC14 merapikan tepi sinyal. Saklar SPDT memakai SR latch.
   - Software debouncing berbasis waktu. Program mencatat millis() setiap sinyal berubah dan memperbarui status hanya jika sinyal stabil lebih lama dari T_{debounce}. Metode ini non-blocking, berbeda dengan delay().
   - Software debouncing berbasis sampling atau counter. Program membaca pin secara periodik, misalnya lewat timer interrupt, dan menerima status baru setelah n sampel berturut-turut bernilai sama.

5. **Cara kerja ADC dan pengaruh resolusi.** ADC mengubah sinyal analog dalam tiga tahap. Tahap sampling dan hold mencuplik tegangan masukan dan menahannya pada kapasitor. Tahap kuantisasi memetakan tegangan ke salah satu dari 2^N level diskrit. Tahap encoding menyimpan level itu sebagai bilangan biner. ADC tipe SAR pada ESP32 melakukan pencarian biner. ADC membandingkan V_{in} dengan keluaran DAC internal bit demi bit dari MSB ke LSB, sehingga satu konversi selesai dalam N langkah.

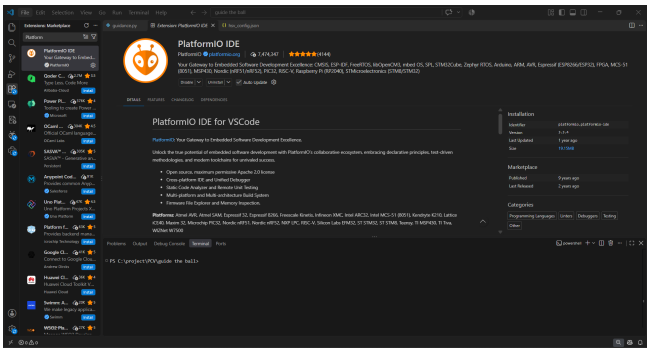
   Resolusi menentukan langkah terkecil yang terbaca:

$$\text{LSB} = \frac{V_{ref}}{2^N}. \tag{4}$$

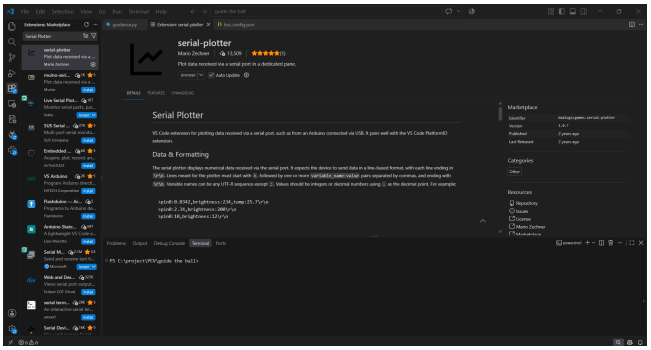
   Dengan $V_{ref} = 3{,}3$ V, LSB ADC 8-bit bernilai sekitar 12,9 mV. LSB ADC 12-bit bernilai sekitar 0,81 mV. Galat kuantisasi maksimum adalah $\pm\frac{1}{2}$ LSB, jadi N yang besar menghasilkan pembacaan lebih presisi. Presisi efektif tetap dibatasi oleh noise, nonlinieritas, dan akurasi tegangan referensi. Ukuran ini dinyatakan sebagai ENOB (Effective Number of Bits). ENOB ADC ESP32 lebih rendah dari 12 bit, jadi kamu perlu mengkalibrasi hasil bacaan atau merata-ratakan beberapa sampel.

6. **Set-up perangkat lunak.** Visual Studio Code dan ekstensi PlatformIO IDE sudah terpasang. Platform Espressif 32 (framework ESP-IDF dan Arduino) dan ST STM32 (framework Arduino/STM32duino)

sudah masuk lewat PlatformIO Home. Ekstensi Serial Plotter sudah terpasang untuk menampilkan data serial, misalnya hasil pembacaan ADC, dalam bentuk grafik. Driver ST-Link dan CP2102/CH340 sudah terpasang, dan sistem operasi mendeteksi kedua board. Tangkapan layar ditampilkan pada screenshot dibawah



Gambar 1: Platform IO Installed



Gambar 2: Serial Plotter Installed